Now run `\d users` to confirm you have indexes "index_users_deleted_email_partial" that you created earlier, and "users_deleted_email" you created just now.

Run the query from earlier again, and confirm the Multicolumn index is used and an Index Only Scan is performed.

Review the trade-offs between these two approaches. Both the Partial index and Multicolumn index made the query much faster. They are both special purpose indexes for certain queries. One trade-off is that they consume different amounts of space and have different maintenance overhead.

Consider the size of the Multicolumn index. Run these SELECT statements from psql to compare the sizes:

```
SELECT
pg_size_pretty(pg_total_relation_size('users_deleted_email'));
 pg_size_pretty
----------------
1360 kB
```

Now check the size of the partial index:

```
SELECT
pg_size_pretty(pg_total_relation_size('index_users_deleted_email_partial'));
 pg_size_pretty
----------------
 16 kB
```

The partial index is much smaller at `16 kB`, but it may be used by fewer queries. You will have to test both of these in your database, carefully analyzing the query performance and the space consumption for each index type.

You've now seen how Partial Indexes can be used as specialized indexes for specific queries, with a small file size.

Next you'll work with a different Index type, the BRIN index, to see how that Index can be used for large result sets while trying to minimize space consumption.

Using BRIN Indexes

You'll use BRIN indexes to help improve query performance on a large data set. BRIN stands for Block Range Index.

What is a block range index entry?